

Using LLMs and Generative AI for Economics Research

Hiro Kasahara

July 5, 2026

Vancouver School of Economics, University of British Columbia

Roadmap

Experience and motivation

The basics: tools and setup

A workflow that works

The binding constraint: performance and tokens

Infrastructure and reproducibility

Experience and motivation

How I got here

- January 10, 2026: drafting a paper from presentation slides.

How I got here

- January 10, 2026: drafting a paper from presentation slides.
- **It worked!**

How I got here

- January 10, 2026: drafting a paper from presentation slides.
- It worked!
- Can I fully automate research to produce a submittable academic research paper every week?

How I got here

- January 10, 2026: drafting a paper from presentation slides.
- It worked!
- Can I fully automate research to produce a submittable academic research paper every week?
- My attempts failed... at least, for now.

How I got here

- January 10, 2026: drafting a paper from presentation slides.
- **It worked!**
- Can I fully automate research to produce a submittable academic research paper every week?
- **My attempts failed...** at least, for now.
- But I learned a lot.

My attempt: a research cycle

1. **Deep research** (ChatGPT / Gemini): retrieve ideas and gaps aimed at top-5 journals, then turn the output into an **actionable plan**.
2. Feed the plan into Claude Code and run “**research-cycle skills**”.

The research cycle, repeated on each piece

Evaluate the contribution (methods, data, plan) → **draft** that piece → **critically re-evaluate** and emit a concrete list of revisions.

Applied 20+ projects. Genuinely **hit and miss**. Strong in mathematical derivations and coding, but **unsupervised research cycle often produces a messy paper** (notations, style, unfocused contribution) which requires complete re-writing.

The auto-generated drafts were not submission-ready at all

Left unattended, the output fails in predictable ways:

- (a) **Hallucinations** in the literature discussion: plausible citations that do not say what is claimed.
- (b) **Cited theoretical results are sometimes wrong**, so propositions rest on incorrect foundations.
- (c) **Manuscripts come out over-complicated and unfocused**: narrative, notation, and style need rewriting from scratch.

I addressed (a) and (b) by creating relevant skills.

Fixing (c) requires my active involvement

→ human is a bottleneck for automation.

Where it works much better: ongoing projects

The cycle pays off most on pre-existing projects:

- Notation, structure, and research directions are already fixed.
- A wrong cited theorem or a hallucinated reference is obvious to me.

Productive uses:

- refining existing results
- cross-checking derivations across independent agents (Codex, Gemini)
- working through a revise-and-resubmit and its referee reports.

The direction is already mine: the leverage is my judgment, not its autonomy.

The basics: tools and setup

The model landscape

Multiple models and versions, with different strengths.

- **Claude Code** — strong agentic orchestration, long-horizon tasks, tool use.
- **Codex** (OpenAI) — strong on math, computation/simulation, code.
- **Gemini / Antigravity** — large context for literature and full-paper scans.

No single model dominates. Use all of them.

- Different versions/models (e.g., Opus 4.7 vs Opus 4.8 vs Fable 5)
- Different effort levels (low, medium, high, xhigh, max, ultracode)

```
/model

Select model
Switch between Claude models. Your pick becomes the default for new sessions. For other...

1. Default (recommended) ✓ Opus 4.8 with 1M context - Most capable for complex work
2. Sonnet Sonnet 4.6 - Best for everyday tasks
3. Sonnet (1M context) Sonnet 4.6 with 1M context - Draws from usage credits
4. Haiku Haiku 4.5 - Fastest for quick answers

+ Ultracode effort +/- to adjust
```

```
/effort

Effort

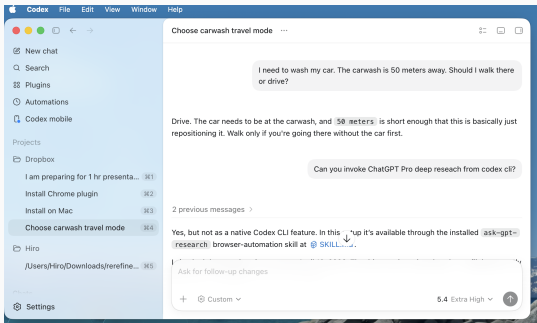
Faster                                     Smarter
low    medium    high    xhigh    max    ultracode
                                     xhigh + workflows

+/- to adjust · Enter to confirm · Esc to cancel
```

Two ways to run them

(a) Desktop / IDE apps

- Native desktop apps, VS Code, Cursor.
- Lower barrier to entry, GUI-driven.



(b) Terminal-based CLIs

- Claude Code CLI, Codex CLI, Antigravity CLI
- Easy to run multiple projects.
- **Installation is easy:** ask claude.ai to get shell commands to terminal

TERMINAL

```
npm install -g @anthropic-ai/claude-code
```

```
cd ~/your-project-folder
```

```
claude
```

My setup: orchestration via subagents or agent team

- Terminal: **Ghostty** + CLIs (Claude Code \$200, Codex \$100, Gemini \$20).
- **Claude Code CLI as the orchestrator** who controls multiple sub/agents.
- **Claude Code calls Codex CLI and Antigravity CLI** as delegated workers **via subagents**.
- Use the best models with the highest effort levels (Opus 4.8[1M]/ultracode, GPT5.5/xhigh, Gemini 3.1 Pro/high)
- Macbook Pro (96GB RAM).

Why subagents/agents – context management / token savings

A subagent runs in its own context window and returns only a summary. The main context stays clean for the work I actually supervise.

A workflow that works

CLAUDE.md and Skills

Two of the most important things to know when using Claude Code:

- CLAUDE.md
- Skills

To be discussed in detail later.

Other things worth knowing:

- `settings.json`
- Plugins / MCP (Model Context Protocol) servers
- hooks

Provide Context and "Ask me any questions" in prompts

The model cannot read your mind. Context gives better judgement. Precise input means fewer corrections.

- Give the context and be concrete about what you want to do:

"I am revising the following file mypaper.tex to make it publishable at AER..."

- Let it interview you using AskUserQuestion tool. I always include

"Ask me any questions"

in my prompt for planning.

Always use "Plan mode" with a mandatory review by Codex

- Shift+Tab $\Rightarrow$ choose "plan mode"
- Give one detailed enough prompt.
- No repeated chats. (Tokens $\uparrow$ exponentially)
- Use **delegate** skill for Codex review ("hook").

Explore $\Rightarrow$ Plan $\Rightarrow$ Codex Review $\Rightarrow$ Implementation

Why "plan mode" with Codex review?

- Quality $\uparrow$
- Token savings
 - Less errors in implementation
 - Context window will be cleared before implementation

Provide examples and styles

When you ask Claude Code to generate an initial draft or presentation slides, give examples and instructions:

- A **previous related paper or slides of yours** as the style target.
- Memos/Slides.
- Explicit **notation and style** conventions.

Supplying concrete examples is the high-return, low effort intervention, for prose and for presentation slides alike.

Revision of your paper

Do *not* ask directly “revise my draft.” Instead:

1. Focus on one issue at a time.
2. Generate a *tex* memo for a copy-pastable, actionable revision plan.
3. “Make it accessible to PhD students.”
4. Revise the paper based on the *tex* memo.

Two trade-offs:

- **scope** $\longleftrightarrow$ **performance**
- **automation** $\longleftrightarrow$ **understanding**

“You can outsource your thinking but not your understanding.” – Andrej Karpathy

Generating presentation slides to deepen understanding

- Once I have a preliminary draft, I typically ask:
“Generate slides for a 1.5-hour seminar: 40 main slides plus 20 backup slides, with cross-references back and forth.”
- It forces you to understand every punchline of the paper.
- Help evaluating what else needs doing and what should be dropped.

A reference library to stop hallucinated citations

The model citing from memory

⇒ fabricated references and mischaracterized results.

- Keep a reference/ folder holding your paper's sources as markdown files.
- Check against the original source text.

How to implement:

- CLAUDE.md
- Skills for citation checks and converting pdf to md files.
 - gemini-pdf (convert pdf to md files)
 - citation-check (hallucinations)
 - download-all-cited-paper (auto-download)

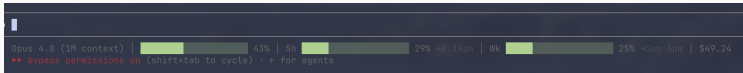
Verification for theory: proofs and derivations

- **Cross-validate** each derivation across multiple *independent* agents (Claude Code, Codex, Gemini).
- Multiple skills for checking proofs:
 - `math-verification` (proofs)
 - `lean-proof-audit` (proofs)
 - `wolfram-verify` (proofs)
- Re-derive from first principles in a fresh context (`proof-from-first-principles`) so the checker is not the author.

The binding constraint: performance and tokens

Context Management and Tokens

1. Usage limits (weekly and 5hrs)



2. **Context window vs. performance.** It holds the entire conversation, every file read, every command output. **Performance degrades as it fills.**

Context management:

1. CLAUDE.md
2. skills
3. /clear
4. plan mode
5. "Bash is all you need"

Read at the start of **every** session. Survives every /clear. Global → Project

- **Standing principles:** plan, verify, ask don't guess. maximum efforts
- **Orchestration:** Use sub/agents. Delegate aggressively to Codex/Gemini.
- **Source-verification rule:** the defense against hallucinated citations.
- Mandatory plan review by codex, memory management, task management etc.

CLAUDE.md affects performance and token consumptions.

My CLAUDE.md (1/7): session start and core principles

```
1  ✓ # CLAUDE.md
2  |
3  ✓ ## Session Start
4
5  1. Read `CHECKPOINT.md` and `MEMORY.md` (if exists) – resume from last known state
6  2. Read `tasks/lessons.md` (if exists) – avoid repeating past mistakes
7  3. Identify active project from working directory and git branch
8  4. If the user references project history, prior decisions, preferences, or "what we did before X", run `bash
   ~/.claude/skills/notebooklm-bridge/scripts/master_brain_query.sh "<topic>"` before answering. Read-only – never uploads.
9
10 ✓ ## Workflow
11
12 ✓ ### Core Principles
13
14 - **Maximum effort**: Apply maximum effort to every task. Think exhaustively, work with exceptional rigor, and carry out each
   assignment thoroughly, carefully, and without compromise. Do not cut corners, skip edge cases, or leave TODOs. Produce complete,
   production-quality output on the first pass by re-reading your output against the original task specification, and fixing any gaps
   or errors before responding.
15 - **Plan before acting**: Plan mode for non-trivial tasks (3+ steps or arch decisions), including verification. Interview me
   exhaustively using AskUserQuestion tool. Specs upfront. If stuck or sideways, STOP and re-plan – don't keep pushing.
16 - **Verify before done**: Prove it works – tests, logs, correctness. Diff behavior between main and your changes when relevant.
   Self-check: "Would a senior engineer approve?"
17 - **Ask, don't guess**: 2-5 targeted questions with concrete options before proceeding under ambiguity.
18 - **Demand elegance**: For non-trivial changes, ask "is there a more elegant way?" If a fix feels hacky: "Knowing everything I know
   now, implement the elegant solution." Challenge your own work before presenting it. Skip for simple fixes.
19 - **Autonomous bug fixing**: When given a bug report: just fix it – don't ask for hand-holding. Point at logs, errors, failing
   tests, then resolve them. Zero context switching required from the user.
```

My CLAUDE.md (2/7): orchestration and routing

```
21 ✓ ### Orchestration
22
23 Operate as manager/orchestrator. **Delegate aggressively, verify rigorously.** Codex ($0, GPT-5.5) and Antigravity ($0, Gemini 3.1 Pro) handle the heavy lifting; Claude Code orchestrates, integrates, and owns the quality gate. Package relevant context into delegation prompts – "needs context" is not a reason to handle directly.
24
25 For trivial fixes (typo, off-by-one, missing import), skip the table and handle directly – delegation overhead isn't worth it.
26
27 **Routing rules** – evaluate in order, first match wins:
28
29 ✓ | # | Condition | Route | Examples |
30 |---|-----|-----|-----|
31 | 1 | Requires iterative local feedback loops (test→fix→retest, multi-file refactor with compile checks) | **Claude Code** | Debugging across files, CI fixes, migration with rollback |
32 | 2 | Secrets, credentials, destructive ops | **Claude Code** | API keys, prod deploys, `rm -rf`, access control |
33 | 3 | High-stakes verification: proofs/lemmas in your own paper, identification arguments | **Claude Code, Codex, Antigravity** | (cross-verify via `ask_both.sh`); independent agent by Claude resolves disagreements | Propositions, theorems, lemmas |
34 | 4 | Routine verification: checking cited results, validating a known proof step | **Antigravity** | "Does Theorem 2 in X actually say Y?", algebra check |
35 | 5 | Code: implement, debug, refactor, test, MC sim, estimation, data wrangling, shell, LaTeX tables/TikZ/Beamer | **Codex** | R/Python scripts, bootstrap CI, GMM routine, booktabs table |
36 | 6 | Reading/reasoning: literature, citations, summarization, deep research | **Antigravity** | Scooping check, citation audit, paper summary |
37 | 7 | Multi-source synthesis (20+ docs), generated artifacts (audio/slides/mind-map), or cross-session memory | **NotebookLM bridge** | Literature survey from 30 PDFs, podcast for commute review, session wrap-up to Master Brain |
38 | 8 | Drafting: prose sections, teaching materials | **Antigravity** (draft) → **Claude Code** (edit for voice/context) | Intro paragraph, lecture notes, problem sets |
39
40 If no row matches cleanly, decompose the task into sub-pieces and route each per the table above.
```

My CLAUDE.md (3/7): subagents

```
42 **Cross-verification third voter**: Row 3 cross-verification with `ask_both.sh` dispatches to Codex and Antigravity. When the two
    disagree, Claude (via the Claude SDK / `ask_claude.sh`) acts as the third voter to resolve disagreements; the orchestrator owns the
    final verdict.
43
44 **PDCA cycle for substantive tasks**
45 1. **Plan** – Decompose; write to `tasks/todo.md` (triggers Codex review hook, max 5 rounds). Present for approval.
46 2. **Do** – Route per table above. Parallel when independent (cap 5 concurrent dispatches). Package context: paste code, spec,
    constraints. One task per subagent.
47 3. **Check** – Verify delegated outputs: (a) check STATUS/VERDICT, (b) run task-type quality gate, (c) for cross-verification,
    resolve disagreements before accepting. Claude Code owns the verdict.
48 4. **Act** – Integrate, document lessons, propose follow-ups.
49
50 Codex plan review hook: after writing `tasks/todo.md`, Codex auto-reviews. Max 5 rounds; if unresolved, present with concerns noted.
51
52 **Delegation failure**: retry once with more specific prompt. After second failure, handle directly and log pattern in
    `tasks/lessons.md`.
53
54 See `/delegate` for scripts, templates, batch dispatch, read discipline, and team patterns.
55
56 ∨ ### Subagent Strategy
57
58 - Use subagents to keep main context window clean. One task per subagent.
59 - Only the orchestrator and its direct subagents may dispatch to Codex/Antigravity (max depth 2). Subagents must not spawn further
    delegating subagents.
60 - Cap: 6 total concurrent external dispatches (Codex + Antigravity combined) across all subagents.
61 - File ownership: when multiple subagents run in parallel, assign non-overlapping file sets.
```

My CLAUDE.md (4/7): workflow, self-improvement, task management

```
63  ✓ ### Five-Phase Coding Workflow
64
65  Use `/five-phase-workflow`. Codex integration: Phase 3 delegates code units to Codex; Phase 4 uses Codex review assist but Claude
    Code owns the quality gate; Phase 5 runs tests locally.
66
67  ✓ ### Self-Improvement
68
69  - After ANY correction: update `tasks/lessons.md` with pattern + prevention rule (not just description).
70  - Write rules for yourself that prevent the same mistake. Ruthlessly iterate until mistake rate drops.
71  - Review lessons at session start.
72
73  ✓ ### Task Management
74
75  1. **Plan First**: Write plan to `tasks/todo.md` with checkable items (PDCA step 1). Interview me exhaustively. Get approval before
    executing.
76  2. **Track Progress**: Keep `tasks/todo.md` current; mark items complete as you go.
77  3. **Explain Changes**: High-level summary at each step.
78  4. **Document Results**: Add review section to `tasks/todo.md` (what worked/didn't).
79  5. **Capture Lessons**: Update `tasks/lessons.md` after corrections.
80
```

My CLAUDE.md (5/7): quality gate, research vault

Quality Gates

Before presenting at Check stage:

Task Type	Must Verify
Code	Runs; tests pass; seeds set; no absolute paths
Proofs	All steps explicit; assumptions stated; convergence modes specified; every cited result verified against original source text (via <code>`citation-check`</code> or <code>`proof-from-first-principles`</code>); algebraic/semialgebraic steps machine-checked via <code>`wolfram-verify`</code> where expressible (Certificate vs Evidence tier recorded; INCONCLUSIVE never read as false)
Empirical	Identification clear; SEs appropriate; diagnostics included
LaTeX	Compiles; citations correct (characterizations added/modified this session verified against source text); follows Writing standards
Delegated	STATUS/VERDICT checked; output matches spec; cross-verification disagreements resolved; integrated result passes task-type gate above

If quality gate fails, retry – do not present to me.

Research vault

Reference library: `/research/reference/`

For research tasks (lit search, citation verification, theorem lookup, reading cited papers), use the ``research-vault-routing`` skill.

Cite as `[[citekey]]`, never as author-year.

My CLAUDE.md (6/7): domain standard

```
104 ✓ ## Domain Standards
105
106 **R/Python simulation**: Cap local cores at ~6. Use alliance-canada skill when estimated runtime >30 min.
107
108 **Math rigor**: State assumptions before theorems. Show non-trivial steps. Notation:  $E$ ,  $\text{Var}$ ,  $\hat{\beta}$ ,  $\mathbf{X}$ . Flag
proof issues. Prove from first principles.
109
110 **Source verification**: cited results in proofs and characterizations of cited work must be checked against original source text in
`~/research/reference/`, not model recall (results provable from definitions alone without citing a specific paper are excluded
unless a specific version is claimed). Locate via `/reference-search`; acquire missing papers via `/download-paper`; for systematic
checks use `/citation-check` or `/proof-from-first-principles`. If source still unavailable, flag explicitly and cross-verify via
`ask_both.sh`. Applies to all agents.
111
112 **Code**: Seeds always set. No absolute paths. `renv`/`requirements.txt` for deps. Comments explain why.
113
114 **Writing & LaTeX**: Concise prose; understate. `natbib` author-year (`\citet`/`\citep`). `booktabs` (no vertical lines); vector
PDF figures. `align` not `eqnarray`; label only if referenced. `amsthm`. Beamer: clean, one idea/slide.
115
116 ✓ ## Communication
117
118 Concise, direct, correct. No filler, caveats, or README files unless requested.
```

My CLAUDE.md (7/7): memory management

```
122 ∨ # MEMORY.md Rules
123
124 - Single source for persistent project knowledge. Structure: `## Stable` / `## Active`. One line per entry, no prose.
125 - Before appending: if >2000 tokens (~150 lines), compact first (merge duplicates, delete resolved, strip context unless it prevents
    repeating mistakes, never duplicate code/config).
126 - After compacting: `<!-- Last compacted: YYYY-MM-DD -->` at top. On session start: if >7 days stale, compact first.
127 - Prune `## Active` aggressively: done/merged = delete.
128
129 ---
130
131 ∨ ## About Me
132
133 Economics professor at UBC. Specializations: econometrics, empirical applied economics, empirical IO. Work spans proofs,
    derivations, empirical research, teaching, refereeing, and mentoring PhD students.
134
135 ∨ ## Languages
136
137 - Default to R unless otherwise specified. For package conventions, see `/r-packages`.
138 - Python for numerical computation when R is insufficient or for validation.
139 - LaTeX for all writing.
140 - Stata only for replication/validation of R results.
```

Examples of my skills

The skills are something you create for repeated use.

1. `delegate` (codex/antigravity cli)
2. `gemini-pdf` (convert pdf to md files)
3. `pdf-to-reference-skill` (convert pdf to md files using Mathpix)
4. `citation-check` (hallucinations)
5. `download-all-cited-paper` (auto-download)
6. `research-vault-routing` (reference library)
7. `math-verification` (proofs)
8. `lean-proof-audit` (proofs)
9. `proof-from-first-principles` (proofs/hallucinations)
10. `style-guide` (writing style template)
11. `refine` (feedback for revision)
12. `aws-spot` (simulation/estimation)
13. `alliance-canada` (simulation/estimation)

How to generate a skill

- Ask the model to generate a skill.
“Generate a `/delegate` skill which calls Codex CLI and Antigravity CLI from Claude Code using subagents...”
- Revise the existing skill to incorporate new features and version updates.

Anatomy

A folder under `~/.claude/skills/`: a `SKILL.md` holding a short description plus the procedure, with optional bash/python scripts the model runs.

Example: the refine skill

Goal: a `refine` skill that reads a `tex/pdf` draft and returns feedback, replicating the service at <https://www.refine.ink/>.

“Generate a `/refine` skill that takes my paper draft and returns a list of feedback, replicating `refine.ink`. Study the service first: browse the site and search the web. Real sample outputs are in `~/Documents/refine-samples/...`”

- Cross-validate the performance: training vs. test data.
- Target: a list of key ground-truth feedbacks in the original refine samples.

Why this prompt works

- **Target:** a concrete service to imitate, not a vague ask.
- **Research:** have the model study the source itself (site + web).
- **Examples:** give real samples of the output you want.

Token discipline: “Bash is all you need” with skills

Tokens are spent reading and writing in the context window. Avoid spending them on things a script can do for free.

- **Zero-token execution**: let the model run a shell command and read three lines of output, not paste a whole file.
- Generate a **skill** for anything you do more than twice.
- Skills contain shell commands, python scripts etc.
- e.g., delegate skill.

Memory and context management

Context is your fundamental resource.

- `/clear` once the 1M context window passes $\sim 40\%$.
- Performance degrades. Token consumptions $\uparrow$ exponentially with $\#$ chats.
- Before clearing, make sure the model knows *what to do next* (`tasks/todo.md`, `memory.md`, `claude-mem` plugin)

The rest of the toolbox: `settings.json`, MCP, plugins, hooks

- `settings.json`: machine-readable config (permissions, model, environment). `CLAUDE.md` is instructions; this is the switchboard.
- **Plugins/MCP (Model Context Protocol)**: connect the model to external tools and data, e.g.,
<https://github.com/openai/codex-plugin-cc>
<https://github.com/thedotmack/claude-mem>
- **Hooks**: shell commands fired at lifecycle events (before/after a tool, on stop) for **deterministic** control: e.g., mandatory Codex review

Infrastructure and reproducibility

Git, GitHub, and worktrees

CLI tools are the most context-efficient way to touch external services. Install `gh`.

- You don't need to know much about Git and Github — Claude Code can take care of it.
- Useful for:
 - Co-authored projects
 - Replication packages:
`https://github.com/hkasahar/kasahara-zhou-jhe-famine`
 - Homepage: `https://hkasahar.github.io/`
- Worktrees: run parallel sessions in isolated checkouts so edits do not collide.

Replication packages

A natural, high-value target, and one with a built-in verification criterion.

- Languages: **R** (default), **Stata**, **Python**, **Matlab**.
- “Prepare an AER replication package per current AEA Data Editor policy and the Social Science Data Editors README template (fetch both first): completed README, Verify on a clean run.”
- Github repos:
 - Replication Package:
e.g., <https://github.com/hkasahar/kasahara-zhou-jhe-famine>
 - R-package: e.g., <https://github.com/bayesian/ak>

Using Computer Clusters

- Submitting jobs to clusters is really easy with Claude Code.
- An example prompt:

“Create a skill named aws-spot for submitting simulation jobs to AWS EC spot instances while minimizing the cost....”

 - `aws-spot`: AWS Spot instances / Batch for long, parallel runs.
 - `alliance-canada`: the HPC cluster in Canada for large jobs.

Other topics

- “Harness Engineering”: design entire operating infrastructure/ecosystems, e.g., MCPs, tool-sharing schemas, multi-agent frameworks, automated feedback loops
- dynamic workflows under ultracode effort level:
 - Claude Code write JavaScripts to orchestrate many sub-agents.
- LLM Knowledge bases:
 - <https://x.com/karpathy/status/2039805659525644595>
 - Notebooklm, Obsidian
- Converting a large pdf file to a md file: Mathpix

From Prompts to Loops: Four Layers of LLM Engineering

Prompt → Context → Harness → Loop

- **Prompt:** write one instruction well *unit: a query*
- **Context:** curate what the model sees
(documents, memory, skills, prior outputs) *unit: the context window*
- **Harness:** equip what the model can do
(tools, permissions, subagents) *unit: the action space*
- **Loop:** structure how the model iterates
(verification, self-correction, delegation, stopping rules) *unit: the workflow*

Verification: the load-bearing idea

- In software, “looks done” is caught by a test or a build.
- In research, a hidden gap in a proof or a consistent-looking but inconsistent estimator survives every automatic check and a confident summary.
- The error is cheap to catch early, expensive once it reaches a final draft or a referee.
- The response:
 - build a pass/fail gate for each checkable item
 - wired into Skills and Hooks so it runs every time.

Verification: example gates against model-specific failures

Mode	Canonical check	Stop rule
Theory, proofs	cross-agent re-derivation; formalization	disagreement halts; theorem must compile
Reduced-form	placebo; known-effect recovery	must vanish on placebo, reappear on truth
Structural, macro	residuals ≈ 0 ; moment match	off target $\Rightarrow$ do not report
Estimation, MC	coverage in band; sandwich SE vs. MC SD	undercoverage $\Rightarrow$ diagnose, never patch

Concluding Remarks

- Use all three models (Claude Code, GPT, Gemini)
 - Different strengths
 - The best models change over time
 - CLAUDE.md and Skills need to be revised
- Security issues (Not discussed)
 - Do not use publicly available skills/MCPs/plugins unless you are really sure.
 - "Auto mode" may provide a good balance between permissions and security.
 - Be careful about handling your API keys.
 - PreToolUse hook: (i) Block writes under your raw data, (ii) Block destructive shell ops: e.g., `rm -rf`.